

UltiMaker NFC-Spulentags

Format, Firmware-Verhalten und SpoolMaker-Konventionen

Technische Zusammenfassung des Reverse Engineerings

Stand	20. August 2026
Untersuchte Firmware	UltiMaker S5 9.0.3-R1; S8 libPalantir aus 11.2.4
SpoolMakerAndroid	v1.1.18 - aktueller Writer/Reader-Stand
Tagtyp	NTAG216, 888 Byte User Memory

Kernaussage

Die UltiMaker-Spule ist ein NTAG216 mit einer vierteiligen NDEF-Nachricht: Materialrecord, Sig-Marker und zwei redundante Statusrecords. Der Drucker bestimmt den aktuellen Status über die höchste akkumulierte Nutzungsdauer - nicht über die verbleibende Menge. SpoolMakerAndroid erhält diese Struktur exakt und nutzt nur die zwei vom untersuchten S5/S8-Code nicht interpretierten Materialbytes für eine eigene Datumskennzeichnung.

Kein offizielles UltiMaker-Dokument. Die Angaben beruhen auf Firmwareanalyse, realen Originaltags und dem implementierten SpoolMakerAndroid-Codec.

Inhalt und Aussageklassen

Dieses Dokument soll die im Projekt gewonnenen Erkenntnisse dauerhaft in einer einzigen technischen Referenz sichern. Es trennt bewusst zwischen direkt beobachteten Daten, Verhalten aus Firmwarecode und eigenen SpoolMaker-Konventionen.

Kennzeichnung	Bedeutung
BESTÄTIGT	Mehrfach an echten Tags beobachtet oder direkt im untersuchten Firmware-/App-Code belegt.
FIRMWARE	Verhalten, das aus dem S5/S8-Code folgt; gilt für die untersuchten Versionen.
HYPOTHESE	Plausible Interpretation, aber nicht durch eine offizielle Spezifikation oder verwendenden Firmwarecode bewiesen.
SPOOLMAKER	Bewusst von SpoolMakerAndroid definierte Konvention. Kompatibilität ist für die untersuchten Firmwarestände plausibel, aber nicht offiziell zugesichert.

Kapitelübersicht

1. Quellenbasis und Geltungsbereich
2. Physischer Tag und NDEF-Gesamtstruktur
3. Materialrecord V0 - Bytebelegung
4. Statusrecord V0 - Bytebelegung und CRC
5. Wie der Drucker die Daten liest und bewertet
6. Warum die beiden Statusrecords nicht identisch sein müssen
7. Das 8-Byte-Zeitfeld: Firmwarebefund und empirische Interpretation
8. SpoolMakerAndroid: exakt geschriebene Felder
9. SpoolMaker-Datumsformat und Datumsarten
10. Reader-Regeln und Integritätsanzeige
11. Praxisbeispiel A - echter UltiMaker-Tag
12. Praxisbeispiel B - mit SpoolMaker geschriebener Tag
13. Grenzen, Risiken und offene Punkte
14. Kanonische Regeln für zukünftige Implementierungen
15. Quellen- und Hash-Anhang

1. Quellenbasis und Geltungsbereich

Die Erkenntnisse stammen nicht aus einer einzelnen Spezifikation, sondern aus dem Abgleich mehrerer unabhängiger Quellen. Entscheidend ist die Übereinstimmung zwischen Firmwareparser, echten Tag-Dumps und dem Verhalten des aktuellen Android-Codecs.

[S5] UltiMaker S5 Firmware 9.0.3-R1 - Datei um-update-9.0.3-R1.swu. Im Firmwareabbild wurde libPalantir unter /usr/lib/python3/dist-packages/libPalantir untersucht; zusätzlich wurde der Controllercode für Auswahl und Aktualisierung der Statusrecords ausgewertet.

[S8] UltiMaker S8 / libPalantir - Vom Benutzer aus S8 Firmware 11.2.4 bereitgestellte libPalantir.zip. Enthält MaterialRecordV0, StatRecordV0, NfcTag, CRC8 und UnitConverter.

Die in dieser Fassung abgedruckten Firmware-Codeauszüge stammen aus genau diesem S8-libPalantir-Paket (11.2.4). Ausgelassene, für die jeweilige Aussage nicht relevante Zeilen werden mit "..." gekennzeichnet.

[TAGS] Originaltags - Mehr als ein Dutzend echte UltiMaker-NTAG216-Dumps aus unterschiedlichen Jahren, Programmierstationen, Chargen und Verbrauchszuständen. Darunter factory-new, fast voll, benutzt und nahezu leer.

[APP] SpoolMakerAndroid - UltimakerTagCodec.java und MainActivity.java, v1.1.18. Enthält den aktuell implementierten Reader, Writer, Datumsmarker und Statusauswahlalgorithmus.

[SM-TAG] SpoolMaker-Testtag - Erfolgreich mit der App geschriebener und zurückgelesener NTAG216, UID 04C9785AD26480, Datum 20.08.2026, Marker 53 03.

Geltungsbereich

Alle Firmwareaussagen beziehen sich auf die konkret untersuchten Versionen. Besonders die Nutzung der beiden Materialbytes 106/107 durch SpoolMaker ist nur deshalb vertretbar, weil diese Bytes in den untersuchten S5/S8-Pfaden nicht interpretiert werden. Eine spätere UltiMaker-Firmware könnte diese Bytes anders verwenden.

Versionsmodell

Die im S8-Paket enthaltenen Material- und Status-Factories kennen ausschließlich Recordversion 0. Der Factorycode kann theoretisch zwischen Recordversion und Kompatibilitätsversion abwärts auf eine unterstützte Klasse zurückfallen; eine V1-Implementierung wurde im bereitgestellten libPalantir nicht gefunden. Originaltags und SpoolMaker schreiben Version 0 / Kompatibilität 0.

2. Physischer Tag und NDEF-Gesamtstruktur

BESTÄTIGT: Die untersuchten Spulen verwenden NTAG216. Der ausgelesene User-Memory-Bereich umfasst 888 Byte, also 222 Seiten zu je 4 Byte von NTAG-Seite 4 bis Seite 225. Die UltiMaker-NDEF-Nachricht beginnt bei den beobachteten Tags direkt am Anfang des User Memory, ohne vorgelagerten Type-2-NDEF-TLV-Wrapper.

FIRMWARE: libPalantir liest zunächst Capability-Container-Daten, prüft die NFC-Forum-Kennung 0xE1 und Major-Version 1 und startet anschließend die NDEF-Auswertung ab Seite 4. SpoolMakerAndroid kann zusätzlich TLV-gepackte NDEF-Nachrichten lesen, schreibt aber das beobachtete rohe Format.

User-Offset	Absoluter Tag-Offset	Record	NDEF-Header	Typ / ID	Payload	Recordlänge
+0	16	Material	0x9C	ultimaker.nl:material / "1"	108 B	134 B
+134	150	Sig	0x11	Sig / -	2 B	8 B
+142	158	Status 1	0x1C	ultimaker.nl:stat / "2"	20 B	42 B
+184	200	Status 2	0x5C	ultimaker.nl:stat / "2"	20 B	42 B

Wichtige Längen

Die logische NDEF-Nachricht ist 226 Byte lang: 134 + 8 + 42 + 42. SpoolMakerAndroid schreibt page-aligned 228 Byte; die letzten zwei Bytes sind dabei 0x00-Padding. Der Rest des 888-Byte-User-Memory gehört nicht zur NDEF-Nachricht.

Die festen absoluten Startindizes 158 und 200 für die beiden Statusrecords sind im S8-NfcTag-Code als Konstanten hinterlegt. Derselbe Code erwartet an diesen Stellen als ersten Headerwert 0x1C bzw. 0x5C und nutzt diese Adressen auch bei der Reparatur eines beschädigten Statusrecords.

3. Materialrecord V0 - Bytebelegung

Der Materialrecord trägt die Identität der Spule und den Materialschlüssel. Der on-wire Payload ist bei allen untersuchten Originaltags 108 Byte lang. Die Firmwareklasse MaterialRecordV0 deklariert jedoch nur 106 Byte als Recordgröße und interpretiert ausschließlich Bytes 0 bis 105.

Firmware-Codebeleg S8 11.2.4 · *libPalantir/contents/material/materialRecordV0.py* · Zeilen 15-19

```
self._serial_number = self._data[2:16]
self._manufacturing_timestamp = struct.unpack(">Q", self._data[16:24])
self._material_id = uuid.UUID(bytes=self._data[24:40])
self._programming_station_id = struct.unpack(">H", self._data[40:42])
self._batch_code = self._data[42:106].decode("utf-8").split("\x00")[0]
```

Belegt: Die Firmware wertet die Nutzdaten nur bis Byte 105 aus. Das 8-Byte-Zeitfeld wird als Big-Endian-uint64 (">Q") eingelesen; eine Umwandlung in Datum oder IEEE-754-double findet in diesem Parser nicht statt. Die auf Originaltags vorhandenen Bytes 106/107 liegen außerhalb dieser V0-Auswertung.

Payload-Offset	Länge	Datentyp	Bedeutung	Firmware / Beobachtung
0	1	uint8	Recordversion	0
1	1	uint8	Kompatibilitätsversion	0
2..15	14	ASCII	Serienfeld	7-Byte-NFC-UID als 14 Hex-Zeichen
16..23	8	raw / >Q	Zeitfeld	Firmwarebezeichnung manufacturing_timestamp ; siehe Kapitel 7
24..39	16	UUID bytes	Material-GUID	Schlüssel zum Materialprofil
40..41	2	uint16 BE	Programmierstations-ID	Originale: verschiedene Stationsnummern
42..105	64	UTF-8, NUL-padded	Batch-/Chargencode	z.B. A2404100283; unterschiedliche Formate beobachtet
106..107	2	nicht interpretiert	Trailing/unknown	Originale: 00 00; SpoolMaker nutzt diese Bytes bewusst

Material-GUID und sichtbare Materialdaten

Der Tag speichert weder Herstellername, Materialname noch Farbe als Klartext. Gespeichert wird die 16-Byte-Material-GUID. Der Drucker und SpoolMaker müssen diese GUID gegen ihre jeweilige Materialbibliothek auflösen. Deshalb importiert SpoolMakerAndroid Cura-Materialdateien: Hersteller, Material, Farbe und gegebenenfalls Spulengewicht bleiben lokal; auf den Tag geht für die Materialidentität nur die GUID.

Serienfeld / UID

BESTÄTIGT: Bei echten Tags entspricht das 14-Byte-ASCII-Serienfeld der 7-Byte-Hardware-UID als Hexstring. Im S5-Code wurde ein Vergleich zwischen diesem Serienfeld und der tatsächlich vom NFC-Chip

gelesenen UID gefunden. SpoolMakerAndroid schreibt daher immer die beim Schreibvorgang gelesene UID und prüft sie beim Zurücklesen erneut.

4. Statusrecord V0 - Bytebelegung und CRC

Jeder Statusrecord hat exakt 20 Payload-Bytes. Er enthält Gesamtmenge, Restmenge und eine akkumulierte Nutzungsdauer. Die beiden Records bilden kein klassisches "Primary + Backup mit identischem Inhalt", sondern zwei alternierend aktualisierte Zustände.

Payload-Offset	Länge	Datentyp	Bedeutung
0	1	uint8	Recordversion = 0
1	1	uint8	Kompatibilitätsversion = 0
2	1	uint8	Einheit: 0 unused, 1 mm, 2 mg, 3 cm ³
3..6	4	uint32 BE	Gesamtmenge
7..10	4	uint32 BE	Verbleibende Menge
11..18	8	uint64 BE	Akkumulierte Nutzungsdauer in Sekunden
19	1	CRC-8	CRC über Bytes 0..18

Einheit 2 bedeutet praktisch Milligramm

Im Firmwarecode heißt die Konstante für Einheit 2 irreführend MATERIAL_QUANTITY_MASS_GR. Die tatsächliche Semantik ist jedoch Milligramm: echte 750-g-Spulen speichern 750000, der UnitConverter dokumentiert seine Masseeingabe als Milligramm, und die Umrechnung nutzt die physikalisch passende mg/mm³-Beziehung. SpoolMakerAndroid bezeichnet Einheit 2 daher korrekt als mg.

Firmware-Codebeleg S8 11.2.4 · statRecordBase.py Zeilen 12-16 + unitConverter.py Zeilen 37-40, 56-59

```
MATERIAL_UNIT_UNUSED = 0
MATERIAL_QUANTITY_LENGTH_MM = 1
MATERIAL_QUANTITY_MASS_GR = 2
MATERIAL_QUANTITY_VOLUME_CM3 = 3

## Convert the input in mass (milligrams) to millimeters filament
def convertFromMassToLength(self, input):
    ...

## Converts length in mm to mass in mg
def convertFromLengthToMass(self, input):
    ...
```

Belegt: Der Unit-Code ist eindeutig 0/1/2/3 = unused/mm/mg/cm³. Der Konstantenname "MASS_GR" ist irreführend, denn der zugehörige UnitConverter bezeichnet Ein- und Ausgabe ausdrücklich als Milligramm bzw. mg. Deshalb steht in der Tabelle nun direkt "2 = mg".

CRC

FIRMWARE: StatRecordV0 berechnet die Prüfsumme über die ersten 19 Bytes und verwirft den Record bei Abweichung. Die verwendete CRC-8 hat das Polynom 0x07 (CCITT), Initialwert 0x00, ohne Final-XOR. SpoolMakerAndroid implementiert dieselbe Berechnung.

```

self._material_unit = self._data[2]
self._material_total = struct.unpack(">L", self._data[3:7])[0]
self._material_remaining = struct.unpack(">L", self._data[7:11])[0]
self._total_usage_duration = struct.unpack(">Q", self._data[11:19])[0]

crc_data = self._data[19]
crc_result = Crc8(self._data[:19]).digest()[0]
if crc_data != crc_result:
    raise ValueError("Crc error on record: %02x %02x" % (crc_data, crc_result))

```

Belegt: Die Bytebelegung des 20-Byte-Statuspayloads folgt direkt aus den Slices. Die CRC liegt in Byte 19 und wird über Bytes 0..18 geprüft; ein abweichender Wert macht den Record für den Parser ungültig.

```

CRC8(data[0..18])
polynomial = 0x07
initial    = 0x00
stored     = payload[19]

```

Nutzungsdauer

BESTÄTIGT/FIRMWARE: total_usage_duration ist ein 64-Bit-Zähler in Sekunden. StatRecordBase beschreibt den Rückgabewert ausdrücklich als akkumulierte Dauer in Sekunden. Beim Drucken sammelt die Firmware temporäre Zeit und addiert sie vor dem Schreiben auf den persistenten Zähler.

Firmware-Codebeleg S8 11.2.4 · statRecordBase.py Zeilen 58-66, 84-87 + crc8.py Zeilen 4-7

```

# @return Returns the accumulated duration in seconds
def getTotalUsageDuration(self):
    return self._total_usage_duration

def prepareForWriting(self):
    self._total_usage_duration += self.__temp_usage_duration
    self.__temp_usage_duration = 0.0

# ... the used polynomial is the CCITT 0x07.

```

Belegt: Die Nutzungsdauer ist explizit ein Sekunden-Zähler und wird vor dem Schreiben um die temporär aufgelaufene Zeit erhöht. Die CRC-Implementierung dokumentiert als verwendetes Polynom CCITT 0x07.

5. Wie der Drucker die Daten liest und bewertet

Die folgende Reihenfolge fasst das beobachtete und aus dem Firmwarecode abgeleitete Verhalten zusammen.

Schritt	Drucker-/Firmware-Verhalten	Folge für Implementierungen
1	NFC-Tag auswählen, Capability Container prüfen, User Memory ab Seite 4 lesen.	Tag muss als kompatibler NFC-Forum-Type-2-Tag erscheinen.
2	NDEF-Records sequenziell parsen. Bekannte Factories behandeln material und stat.	Reihenfolge und NDEF-Flags müssen stimmen.
3	Materialrecord V0 lesen: UID-Serienfeld, Zeitfeld, GUID, Station, Batch.	Material-GUID ist der zentrale Materialschlüssel.
4	Statrecord nur akzeptieren, wenn Recordversion unterstützt und CRC gültig ist.	Ein beschädigter Status kann aus der Recordliste verschwinden.
5	Aktuellen Zustand anhand der größten total_usage_duration wählen.	Restmenge darf NICHT zur Altersbestimmung genutzt werden.
6	Beim Update den älteren Statusrecord mit der kleineren Nutzungsdauer überschreiben.	Es entsteht ein alternierendes Journal aus zwei Zuständen.

Schritt	Drucker-/Firmware-Verhalten	Folge für Implementierungen
7	Bei genau einem gültigen Statusrecord kann libPalantir diesen auf die feste Gegenposition kopieren.	Ein einzelner guter Record ist prinzipiell reparierbar.

Aktiver Statusrecord

S5 9.0.3-R1 wählt den neuesten Record mit `max(stat_records, key=getTotalUsageDuration)`. Beim Schreiben wird der ältere mit `min(...)` ausgewählt. Bei gleicher Nutzungsdauer liefert Python den zuerst vorkommenden Record. Damit gewinnt bei Gleichstand Statusrecord 1.

Reparaturverhalten

Im S8-NfcTag-Code ist eine Reparaturfunktion enthalten. Sind beide Statusrecords unbrauchbar, meldet sie, dass keine Wiederherstellung möglich ist. Ist genau ein Record gültig, wird dessen komplette NDEF-Struktur auf die fest definierte Gegenposition kopiert und der jeweilige Headerwert 0x1C bzw. 0x5C gesetzt.

Firmware-Codebeleg S8 11.2.4 · `libPalantir/nfcTag.py` · Zeilen 31-38, 159-189 (gekürzt)

```
__START_INDEX_RECORD_0 = 158
__START_INDEX_RECORD_1 = 200
__START_INDEX_VALUE_RECORD_0 = b"1C"
__START_INDEX_VALUE_RECORD_1 = b"5C"

stat_records = self.getRecordsByType(StatRecordFactory.getType())
if len(stat_records) == 1:
    ...
    data = ndef_container.getRawData()
    new_ndef_container.addData(data)
    new_ndef_container.setHeaderByte(hex_value=new_start_index_value)
```

Belegt: Die Firmware kennt die beiden festen Recordstarts 158 und 200 und deren Headerwerte 0x1C/0x5C. Wenn nur ein gültiger Statusrecord übrig ist, kopiert die Reparaturlogik dessen vollständige NDEF-Daten an die Gegenposition und setzt den passenden Header.

Sig-Record

BESTÄTIGT: Zwischen Material und Status liegt ein NDEF-Well-Known-Record vom Typ "Sig" mit Payload 20 00. In den untersuchten libPalantir-Pfaden existiert dafür keine Material-/Stat-Factory und es wurde keine kryptographische Verifikation dieses Payloads gefunden. Der Wert ist deshalb als Format-/Kompatibilitätsmarker zu behandeln, nicht als Nachweis der Echtheit. SpoolMakerAndroid schreibt denselben Marker, zeigt ihn aber nur informativ an.

6. Warum die beiden Statusrecords nicht identisch sein müssen

Mehrere Originaltags widerlegen die Annahme, dass die beiden Statusrecords bytegleich sein müssten. Der Drucker aktualisiert sie abwechselnd. Zusätzlich kann die gespeicherte Restmenge zwischen zwei zeitlich aufeinanderfolgenden Zuständen sogar steigen.

Beispiel	Status 1: Rest / Zeit	Status 2: Rest / Zeit	Aktiv	Beobachtung
Original UID 04758ECA906B81	972 mg / 423097 s	7862 mg / 423098 s	2	Neuerer Record hat 6890 mg MEHR Rest.
Original, benutzt	491138 mg / 91207 s	498028 mg / 91208 s	2	Neuerer Record hat mehr Rest.
Original, fast leer	57770 mg / 496680 s	57840 mg / 496652 s	1	Status 1 ist wegen höherer Zeit aktueller.
Original, fast voll	739034 mg / 4353 s	739411 mg / 4354 s	2	Erneut steigt der Rest im neueren Record.
Gleichstand beobachtet	unterschiedlich / gleiche Zeit	unterschiedlich / gleiche Zeit	1	Python-max-Tie: erster Record gewinnt.

Warum kann die Restmenge steigen?

Die Firmware arbeitet beim Drucken intern mit Filamentlänge und Materialparametern wie Dichte und Durchmesser. StatRecordBase konvertiert die verbleibende Länge zurück in die gespeicherte Masse. Rundungen, Schätzkorrekturen oder aktualisierte Verbrauchsmodelle können daher zu einem leicht höheren gespeicherten Massenwert führen, obwohl die Zeit weiterläuft. Die Nutzungsdauer ist deshalb der belastbare Reihenfolgeindikator.

Konsequenz für Integritätsprüfung

"Statusrecords bytegleich" ist höchstens eine Information. Abweichende Records sind bei gebrauchten Spulen normal und dürfen nicht als Integritätsfehler markiert werden. Entscheidend sind gültige CRCs und die Auswahl des Records mit der höchsten Nutzungsdauer.

7. Das 8-Byte-Zeitfeld: Firmwarebefund und empirische Interpretation

Dieses Feld ist der wichtigste Punkt, bei dem Firmwarebezeichnung und beobachtete Daten auseinanderlaufen. Deshalb muss zwischen gesichertem Befund und Interpretation unterschieden werden.

Ebene	Befund
FIRMWARE	MaterialRecordV0 nennt das Feld manufacturing_timestamp und liest Bytes 16..23 mit struct.unpack(">Q", ...), also als Big-Endian-uint64-Bitmuster. Im untersuchten Druckerpfad wird daraus kein Datum berechnet und das Feld beeinflusst den Druck nicht.
BEOBACHTUNG	Werden dieselben 8 Bytes stattdessen als Big-Endian IEEE-754 double interpretiert, ergeben viele Originaltags sehr plausible Unix-Sekundenwerte, passend zu Produktions-/Kaufzeiträumen.
GEGENBEISPIELE	Einige Programmierstationen liefern so interpretiert Werte nahe 1970. Das spricht eher für falsch gesetzte oder uninitialisierte Stationsuhren als gegen das Double-Modell, beweist dieses aber nicht.
HYPOTHESE	Das Produktionssystem hat sehr wahrscheinlich einen double mit Unix-Sekunden in diese 8 Bytes geschrieben. Für die Druckerfunktion ist diese Semantik unerheblich, weil die untersuchte Firmware das Feld nicht als Zeit verwendet.

Repräsentative Originalwerte

Raw Hex	Als BE-double	Interpretation	Kommentar
41D816F0B45BA776	1616626385.432096	24.03.2021 23:53:05	Original UID 04758ECA906B81, Station 42
41D986A0D10EE06E	1713013572.232448	13.04.2024	Factory-new Tag; Batch A2404100283
41D96D05BA942BED	1706301162.315181	26.01.2024	Plausibel zu späterem Kaufdatum
40F517FECEBDD335	86399.925...	01.01.1970 23:59:59	Beispiel für offensichtlich falsche Stationszeit

Nicht als offizielles Herstellungsdatum behaupten

Bei Originaltags zeigt SpoolMakerAndroid deshalb bewusst "UltiMaker-Zeitfeld" und zusätzlich Raw-Hex, Raw-uint64, Double-Wert und die daraus berechnete Datum/Zeit-Interpretation. Die App bezeichnet diesen Wert bei Originaltags nicht verbindlich als Herstellungsdatum.

8. SpoolMakerAndroid: exakt geschriebene Felder

SPOOLMAKER: Der Writer erzeugt bewusst dieselbe NDEF-Geometrie wie die Originaltags. Es gibt genau einen Writer-Einstiegspunkt. Die UID wird beim Schreibvorgang vom realen NFC-Tag übernommen; Material-GUID, Gesamtgewicht, Restgewicht und Datumsoption kommen aus der App.

Feld	SpoolMaker-Wert	Begründung / Wirkung
Material Version / Compat	0 / 0	Entspricht den untersuchten Originaltags und dem S5/S8-V0-Parser.
Serienfeld	Hardware-UID, 14 Hex-Zeichen	Erfüllt die UID/Serienfeld-Konsistenz.
Zeitfeld	BE IEEE-754 double Unix-Sekunden; oder 0.0	Eigene Datumssemantik, siehe Kapitel 9.
Material-GUID	Aus gewähltem Cura-Materialprofil	Nur die GUID wird auf dem Tag gespeichert.
Programmierstation	0xAFFE	Bewusste Kennung für SpoolMaker-erzeugte Tags.
Batchcode	123456789AB, NUL-padded auf 64 Byte	Aktueller synthetischer Platzhalter.
Bytes 106/107	0x53 + Datumsart-Code	SpoolMaker-Erweiterung in den von S5/S8 nicht interpretierten Bytes.
Sig-Payload	20 00	Kompatibilitätsmarker wie bei Originaltags.
Status 1 + 2	identische initiale V0-Records	Drucker kann danach alternierend weiterschreiben.
Status-Einheit	2 = mg	Gesamt- und Restgewicht in Milligramm.
Nutzungsdauer	0 s	Neue Spule startet ohne Nutzungshistorie.
CRC	CRC-8 0x07 je Statusrecord	Wird über Bytes 0..18 berechnet.

NDEF-Template

```
Record 1  0x9C  ultimaker.nl:material  ID="1"  payload=108
Record 2  0x11  Sig                    ID=-   payload=20 00
Record 3  0x1C  ultimaker.nl:stat    ID="2"  payload=20
Record 4  0x5C  ultimaker.nl:stat    ID="2"  payload=20

logical NDEF length = 226 bytes; write buffer = 228 bytes (4-byte aligned)
```

SpoolMaker-Erkennung

Die App erkennt einen eigenen Tag nur dann als SpoolMaker-Format, wenn gleichzeitig die Programmierstations-ID 0xAFFE ist und Byte 106 den Marker 0x53 enthält. Dadurch reicht weder eine zufällige Stationsnummer noch ein einzelnes Markerbyte allein für die Klassifikation.

9. SpoolMaker-Datumsformat und Datumsarten

SPOOLMAKER: Die zwei trailing Bytes werden als minimale Erweiterung verwendet. Byte 106 ist ein fester Marker; Byte 107 beschreibt die Bedeutung des im 8-Byte-Zeitfeld gespeicherten Datums.

Byte 106	Byte 107	Enum	Bedeutung	Zeitfeld
0x53	0x00	NONE	Kein eigenes Datum	0.0
0x53	0x01	MANUFACTURED	Herstellungsdatum	gewähltes Datum
0x53	0x02	PURCHASED	Kaufdatum	gewähltes Datum
0x53	0x03	OPENED	Geöffnet am	gewähltes Datum

Byte 106	Byte 107	Enum	Bedeutung	Zeitfeld
0x53	0x04	CREATED	Spule angelegt am	gewähltes Datum

Speicheralgorithmus

1. Der Benutzer wählt eine Datumsart und ein Kalenderdatum.
2. Die App setzt dieses Kalenderdatum auf 00:00:00 UTC. Damit wird bewusst nur ein Datum, keine lokale Uhrzeit gespeichert.
3. Aus dem UTC-Zeitpunkt werden ganze Unix-Sekunden berechnet.
4. Die Sekunden werden als IEEE-754 double dargestellt und dessen 64-Bit-Bitmuster Big-Endian in Materialbytes 16..23 geschrieben.
5. Byte 106 wird auf 0x53 gesetzt; Byte 107 enthält den Datumsart-Code.
6. Bei NONE wird das Zeitfeld 0.0 und der Marker 53 00 geschrieben.

Warum 00:00 UTC?

Die gespeicherte Information ist ein Kalenderdatum. UTC-Mitternacht vermeidet Verschiebungen durch Zeitzonen und Sommerzeit. SpoolMaker zeigt eigene Datumswerte deshalb als Datum in UTC an. Die Altersberechnung arbeitet ebenfalls auf Tagesbasis.

Konkretes Beispiel: 20.08.2026 / "Geöffnet am"

Eigenschaft	Wert
Unix-Sekunden	1787184000
Double	1787184000.0
Raw-Bits Big-Endian	41 DA A1 90 E0 00 00 00
Station	AF FE
Trailing Bytes	53 03

10. Reader-Regeln und Integritätsanzeige

Die App wurde nach den Firmware- und Tagbefunden so geändert, dass sie echte gebrauchte Spulen korrekt bewertet und nicht mehr fälschlich identische Statuskopien oder den Sig-Wert als Echtheitskriterium verlangt.

Prüfung / Anzeige	Aktuelle Regel
Aktiver Status	Unter CRC-gültigen Records gewinnt die höchste total_usage_duration. Bei Gleichstand bleibt der frühere Record, also Record 1.
Wenn alle CRCs ungültig	Nur diagnostischer Fallback auf höchste Nutzungsdauer; dies ist keine Aussage, dass der Drucker den Tag akzeptiert.
Statuscopies bytegleich	Nur Information, kein Fehlerkriterium.
Status-CRC	Für jeden Record separat berechnet und angezeigt.
UID gegen Serienfeld	Wird als Konsistenzprüfung ausgewiesen.
Sig 0x2000	Als vorhandener Marker angezeigt, nicht als kryptographische Authentizität.
Original-Zeitfeld	Raw Hex + uint64 + BE-double + Datum/Zeit-Interpretation.
SpoolMaker-Zeitfeld	Datumsart + Datum + Alter seit Datum.

Reader-Robustheit

SpoolMakerAndroid liest sowohl das bei UltiMaker beobachtete rohe NDEF ab User-Memory-Offset 0 als auch einen NFC-Forum-Type-2-TLV-Wrapper. Für Originaltags wird außerdem die volle 888-Byte-User-Memory-Struktur ausgelesen und kann als Hexdump exportiert werden.

11. Praxisbeispiel A - echter UltiMaker-Tag

Der folgende Tag ist ein sehr gutes Referenzbeispiel, weil die Restmenge im neueren Statusrecord deutlich höher ist und damit die falsche "kleinster Rest = neuester Record"-Heuristik klar widerlegt.

Eigenschaft	Wert
Chip-UID / Serienfeld	04758ECA906B81 / 04758ECA906B81
Material-GUID	40a273c6-0e15-4db5-a278-8eb0b4a9e293
Programmiersation	0x002A (42)
Batch	17291602
Zeitfeld Raw	41D816F0B45BA776
Zeitfeld als BE-double	1616626385.432096 s -> 24.03.2021 23:53:05
Trailing Bytes	0000
Sig	2000

Status	Gesamt	Rest	Nutzungsdauer	CRC	Bewertung
1	750000 mg	972 mg	423097 s	0x9A gültig	älter
2	750000 mg	7862 mg	423098 s	0x26 gültig	AKTIV

Interpretation

Status 2 ist eindeutig aktueller, weil 423098 > 423097. Dass er 7,862 g statt 0,972 g Rest enthält, ist kein Widerspruch zum Firmwaremodell. Es ist vielmehr ein reales Beispiel dafür, warum allein die Nutzungsdauer die Reihenfolge bestimmt.

12. Praxisbeispiel B - mit SpoolMaker geschriebener Tag

Dieser Tag wurde mit SpoolMakerAndroid v1.1.18 beschrieben und unmittelbar bytegenau zurückgelesen. Er belegt den Writer-Aufbau einschließlich eigener Datumsmetadaten.

Eigenschaft	Wert
Chip-UID / Serienfeld	04C9785AD26480 / 04C9785AD26480
Material-GUID	ca27d214-ea51-418e-a43f-7f82e319d814
Programmiersation	0xAFFE
Batch	123456789AB
Datumsart	OPENED / Geöffnet am
Datum	20.08.2026
Zeitfeld Raw	41DAA190E0000000
Zeitfeld Double	1787184000.0 s

Eigenschaft	Wert
Trailing Bytes	5303
Sig	2000

Status	Gesamt	Rest	Nutzungsdauer	CRC	Aktiv
1	1000000 mg	1000000 mg	0 s	0x5D gültig	ja (Tie -> erster)
2	1000000 mg	1000000 mg	0 s	0x5D gültig	nein

Die logische NDEF-Länge beträgt auch hier 226 Byte. Der physische Schreibpuffer ist 228 Byte lang und endet mit zwei Nullbytes. Danach blieb der restliche NTAG216-User-Memory im Testtag null.

Stand der realen Kompatibilitätsprüfung

Bestätigt ist bisher: Schreiben mit Android, bytegenaues Rücklesen, korrekte CRCs, korrekte NDEF-Geometrie und Erkennung als SpoolMaker-Tag. Noch separat zu verifizieren ist, wie S5/S8 den selbst erzeugten Tag nach einem realen Druck aktualisieren und ob die Bytes 53 xx dabei unverändert bleiben.

13. Grenzen, Risiken und offene Punkte

Punkt	Status / Risiko
Offizielle Spezifikation	Nicht verfügbar. Die Dokumentation ist Reverse Engineering.
Zeitfeld-Semantik	Double/Unix-Sekunden ist stark empirisch gestützt, aber die Druckerfirmware selbst interpretiert das Feld nicht als double und nutzt es nicht als Datum.
Bytes 106/107	In den untersuchten S5/S8-Versionen ungenutzt; zukünftige Firmware kann sie definieren.
Programmierstation 0xAFFE	Eigene SpoolMaker-Kennung; keine UltiMaker-Zuweisung bekannt.
Batch 123456789AB	Synthetischer Platzhalter, nicht aus einer realen Herstellungsladung abgeleitet.
Sig 20 00	Kein gefundener kryptographischer Echtheitsnachweis. Nur Formatmarker in dieser Dokumentation.
Materialverfügbarkeit	Ein formal korrekter Tag hilft nur, wenn die Material-GUID vom Drucker bzw. dessen Materialprofilen verstanden wird.
Firmwareumfang	Aussagen gelten für S5 9.0.3-R1 und die bereitgestellte S8-libPalantir-Version aus 11.2.4.

Besonders wichtig für zukünftige Änderungen

- Die Offsets der Statusrecords nicht verschieben. Deshalb bleibt der Material-Payload physisch 108 Byte lang.
- Die beiden Statusrecords nicht zwangsweise gleichsetzen oder beim Lesen "reparieren". Das ist Aufgabe des Druckers/Material-Station-Pfads, nicht des normalen Readers.
- Den aktiven Status nie anhand der Restmenge bestimmen.
- Original-Zeitfelder nicht automatisch in SpoolMaker-Metadaten umschreiben.
- Markerbytes nur bei bewusst neu erzeugten SpoolMaker-Tags setzen.
- Bei Änderungen am Writer immer einen Roundtrip- und einen echten Tag-Smoke-Test durchführen.

14. Kanonische Regeln für zukünftige Implementierungen

Diese Regeln sind die kurze, normative Zusammenfassung dessen, was ein kompatibler SpoolMaker-Reader/Writer aus heutiger Sicht tun soll.

Reader

1. 888 Byte User Memory lesen, NDEF entweder roh oder TLV-gekapselt erkennen.
2. Materialrecord und alle Statusrecords parsen; Sig separat nur als Marker erfassen.
3. Material-Payload ab 106 Byte akzeptieren; bei 108 Byte die beiden trailing Bytes erhalten/anzeigen.
4. CRC jedes Statusrecords über seine ersten 19 Bytes prüfen.
5. Für Nutzdaten zuerst CRC-gültige Statusrecords betrachten; höchste total_usage_duration gewinnt; Gleichstand -> erster Record.
6. Statusgleichheit niemals als notwendige Integritätsbedingung verwenden.
7. UID und Serienfeld vergleichen.
8. Original-Zeitfeld immer roh sichern; Double-/Unix-Datum nur als Interpretation kennzeichnen.
9. SpoolMaker-Datum nur auswerten, wenn Station=0xAFFE und Markerbyte=0x53.

Writer

1. Materialrecord: Version 0, Compat 0, Serienfeld = echte Hardware-UID.
2. Material-Payload exakt 108 Byte; GUID standardmäßig als 16 UUID-Bytes.
3. Station 0xAFFE; Batch aktuell 123456789AB; trailing Bytes 0x53 + DateMeaning.
4. Datum als UTC-Mitternacht -> Unix-Sekunden -> IEEE-754 double -> Big-Endian-Bits; NONE -> 0.0.
5. Sig-Record mit 20 00 schreiben.
6. Zwei identische initiale Statusrecords: Version 0, Compat 0, Einheit 2, total/rest in mg, usage=0, korrekte CRC.
7. NDEF-Recordstarts bei User-Offsets 0, 134, 142, 184 beibehalten; logische Länge 226 Byte.
8. Nach dem Schreiben denselben Tag zurücklesen und mindestens UID, GUID, Gewichte, CRCs, Station, Marker und Datum verifizieren.

```
Material payload (108 B)
00 00 | <UID ASCII 14> | <time 8> | <UUID 16> | AF FE | <batch 64> | 53 <meaning>

Status payload (20 B)
00 00 02 | <total u32 BE> | <remaining u32 BE> | <usage u64 BE> | <CRC8>
```

15. Quellen- und Hash-Anhang

Die folgenden Hashes fixieren die konkret verwendeten Artefakte. Damit kann später überprüft werden, ob dieselben Quelldateien vorliegen.

Die abgedruckten Python-Auszüge sind kurze, unveränderte Ausschnitte aus dem gehashten S8-libPalantir-Archiv; "..." markiert lediglich ausgelassene Zeilen.

Artefakt	Datei	SHA-256
S5 Firmware 9.0.3-R1	um-update-9.0.3-R1.swu	6b10c4fad5e6b89b40fef72bf06eee352d0828c00a69f938f44af443476cf81c
S8 libPalantir	libPalantir.zip	743362d918ebe4377ac1ded3875b0d5ec80134fb558300b32adeda344b9bf7af
Codec v1.1.18	UltimakerTagCodec.java	2d6b0816dfe3cb22ae51f43f4a02d1e551c2a82668743a0a42c52a98343b29eb
MainActivity cosmetics/date UI	MainActivity.java	73977e9e7487f802182d5eaa65fd8fa1dbeb12731fe88bd827cb72bcca40c
Aktuell zwischengeparkte APK	app-debug(3).apk	031655470f9d3f9d9205245597cedaef1bd95f107631269480fc39beaffd31e2

Wichtige Codepfade im S8-libPalantir

Pfad	Relevanz
libPalantir/contents/material/materialRecordV0.py	106 interpretierte Materialbytes; UID, >Q-Zeitfeld, UUID, Station, Batch.
libPalantir/contents/material/materialRecordFactory.py	Nur MaterialRecordV0 registriert.
libPalantir/contents/stat/statRecordV0.py	20-Byte-Statuslayout und CRC-Prüfung.
libPalantir/contents/stat/statRecordBase.py	Einheiten, Sekunden-Zähler, Längen-/Massenkonvertierung.
libPalantir/math/crc8.py	CRC-8-Tabelle, Polynom 0x07.
libPalantir/nfcTag.py	NDEF-Lesen; feste Stat-Startindizes 158/200; Reparaturmechanismus.

SpoolMakerAndroid Repository

<https://github.com/joker-mik/SpoolMakerAndroid>

Abschluss

Der zentrale technische Befund ist stabil: Das UltiMaker-Tagformat ist ein kleines, redundantes Journal. Der Materialrecord identifiziert die Spule und das Material; die beiden Statusrecords bilden zeitlich geordnete Zustände, wobei die akkumulierte Nutzungsdauer die Reihenfolge bestimmt. SpoolMakerAndroid verändert dieses Grundmodell nicht, sondern ergänzt lediglich eine klar markierte Datumssemantik in zwei bislang ungenutzten Materialbytes.